

AGORA

An agent based market simulation engine

Architecture and Mathematical Design Specification

*A tick by tick limit order book, populated by heterogeneous trading agents,
built for learning, showcase, and later strategy analysis.*

Investment portfolio . Quantitative analysis

Version 0.1 . Design phase . August 2026

Contents

1 Purpose and scope.....	1
1.1 What Agora is not.....	1
1.2 Design goals.....	1
2 System architecture.....	1
3 Module map	1
4 Core data model	1
5 Matching engine	1
5.1 Price and time priority	1
5.2 Discretisation, tick and lot	1
5.3 Order types.....	1
5.4 Auctions	1
5.5 Order lifecycle and cancellations.....	1
6 The simulation clock	1
7 Agent framework.....	1
8 Agent archetypes and their mathematics	1
8.1 Noise trader	1
8.2 Market maker, the Avellaneda and Stoikov model	1
8.3 Momentum trader.....	1
8.4 Value investor	1
8.5 The remaining archetypes	1
9 Environment layer	1
9.1 Fundamental value	1
9.2 News and shocks	1
9.3 Circuit breakers.....	1
9.4 Corporate actions and supply.....	1
10 Frictions	1
11 Leverage, margin, and liquidation	1
12 Validation framework.....	1
13 Calibration	1
14 Analytics and metrics.....	1
15 Reproducibility and ensembles	1
16 Technology and performance.....	1
17 Project structure.....	1
18 Roadmap.....	1
19 Risks and open questions	1
20 What to build first.....	1
Appendix A Parameter reference	1
Appendix B Notation	1

Fields not showing. Right click the table above and choose Update Field to build the page list.

1 Purpose and scope

This document sets out the design for Agora, an agent based market simulation engine. Agora models a single traded asset at the level of individual orders. It runs a continuous time limit order book, filled by software agents that each follow a trading strategy. Prices are not drawn from a formula. They emerge from the orders those agents send and the trades the engine matches against them.

The engine has two jobs. First it is a learning and showcase piece. It brings quantitative modelling, clean software design, and market microstructure knowledge together in one place. Second it is the base for later work on strategy analysis. Once the engine reproduces real market behaviour, a trading strategy can be dropped in and tested against a market that reacts to it, rather than against a fixed historical tape that does not.

1.1 What Agora is not

It helps to be honest about the boundaries. Agora is not a portfolio return projector. It does not forecast where a real stock will trade next year. It is not a backtester against historical tick data, although it can sit next to one and be compared. It is not a production trading system. Latency and speed are modelled for realism, not optimised to trade real money.

1.2 Design goals

- **Realism first.** every run is scored against the empirical stylised facts before it is trusted
- **One asset done well.** before many assets are added
- **Clean separation.** the market, the agents, and the analysis are three different things and stay that way
- **Reproducible.** the same seed gives the same path, every time
- **Fast enough.** to run hundreds of paths, not just one

2 System architecture

Agora has three layers. Each layer owns one kind of decision. The layer above drives the layer below. The market state produced at the bottom feeds back to the top on the next step, which is what closes the loop and lets prices form on their own.

System architecture

Three layers. The top drives the layer below, market state feeds back up.

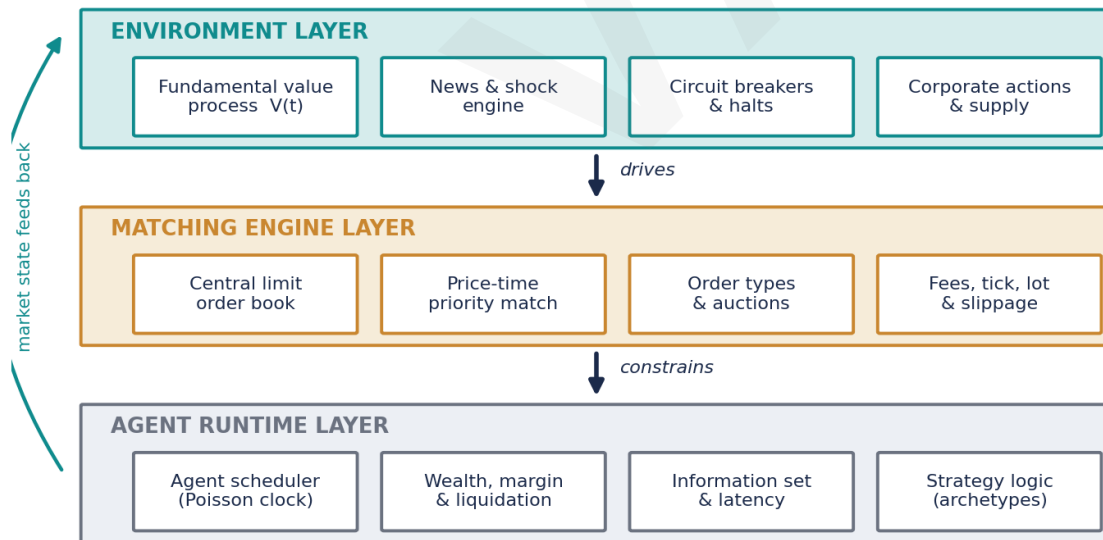


Figure 1. The three layer stack. Top drives bottom, state feeds back up.

Environment layer. This sets the world the agents trade in. It holds the true value of the asset, the news process that moves that value, and the exchange rules that can pause trading. Agents never see the true value directly. They infer it, or ignore it, depending on their strategy.

Matching engine layer. This is the exchange. It keeps the order book, matches buy and sell orders by a fixed set of priority rules, and applies the frictions of a real venue. Tick size, lot size, fees, and slippage all live here.

Agent runtime layer. This runs the traders. It decides who acts and when, builds each agent's view of the market, applies latency, and enforces each agent's budget. When an agent runs out of money or breaches its margin, this layer liquidates it.

3 Module map

The code follows the layers. One module owns each concern, and dependencies point inward towards the core. Nothing in the book knows about a specific agent, and no agent reaches into the matching logic. This keeps each piece testable on its own.

Module map

One clear owner per concern. Dependencies point inward to the core.

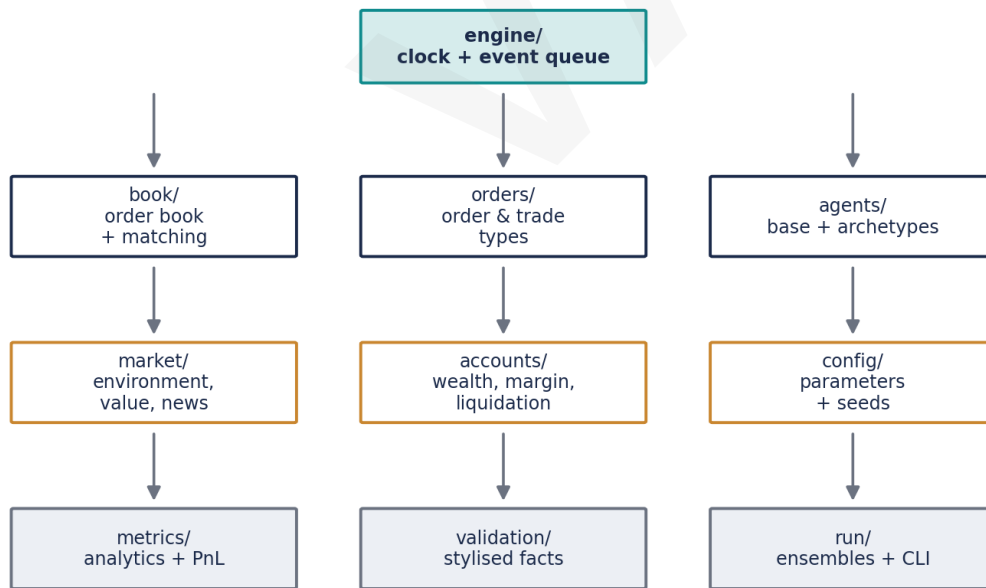


Figure 2. Module map. Arrows are dependencies, pointing towards the core.

Module	Owns	Depends on
engine	The clock and the time ordered event queue	book, agents, market
book	The order book and the matching rules	orders
orders	Order and trade objects, plus order types	nothing
agents	The base agent and every archetype	orders, accounts
market	Environment, value process, news, halts	orders
accounts	Wealth, margin, and liquidation	orders
config	Parameters and seeds	nothing
metrics	Analytics and profit and loss	book, orders
validation	Stylised fact score card	metrics
run	Ensembles and the command line entry	everything

4 Core data model

Five objects carry the whole simulation. Keeping them small and explicit is what keeps the engine auditable. If a figure looks wrong later, it can be traced back through these five to a single order.

Object	Key fields
Order	id, agent id, side, type, limit price, size, visible size, time, time in force
Trade	buy order, sell order, price, size, time, which side was the taker
OrderBook	bids and asks as price ordered queues, best bid, best ask, depth by level
Account	cash, inventory, borrowed, equity, realised and unrealised profit
Agent	strategy and its parameters, account, latency, last seen market view

5 Matching engine

The matching engine is the exchange. A simple price crossing loop is not enough, because the physical rules of a real venue are what create queues, spreads, and slippage. Those rules are modelled here.

5.1 Price and time priority

The best price always wins. Among orders resting at the same price, the one that arrived first is filled first. This is the standard rule on most equity venues and it is easy to reason about. Pro rata allocation, where fills are split across orders by size, is noted as an option for a later derivatives mode but is not the default.

5.2 Discretisation, tick and lot

Prices move in whole ticks. Sizes move in whole lots. A tick of one penny and a lot of one share are the defaults. This matters more than it looks. Discrete prices are what force many orders onto the same level, and that is what creates a real queue with real time priority.

5.3 Order types

Limit and market orders come first. The engine then adds four more, because each one changes behaviour in a way that matters.

- **Immediate or cancel.** fills what it can at once and cancels the rest
- **Fill or kill.** fills in full at once or does nothing
- **Stop orders.** turn into market orders when a trigger price is hit, which is how a falling price creates more selling
- **Iceberg orders.** show only part of their size, which changes how other agents read the book

5.4 Auctions

Continuous trading does not run all the time. The day opens and closes with a call auction. Orders collect for a short window, then cross once at the single price that clears the most volume. Opening prices in real markets are set this way, so an engine that skips the auction misses how each session starts.

5.5 Order lifecycle and cancellations

Real venues see far more cancellations than trades. Often more than nine in ten resting orders are pulled before they ever fill. Agents must be able to cancel and replace as the price moves away from their quote. The engine tracks the full lifecycle of every order, so the cancellation to trade ratio can be measured and tuned to a realistic level.

6 The simulation clock

Agora runs in continuous time, not in fixed rounds. Every future action sits in a time ordered queue. The engine always jumps to the next event, whatever it is, and advances the clock to that moment. This is more faithful than waking every agent on every tick, and it lets fast and slow agents live in the same market.

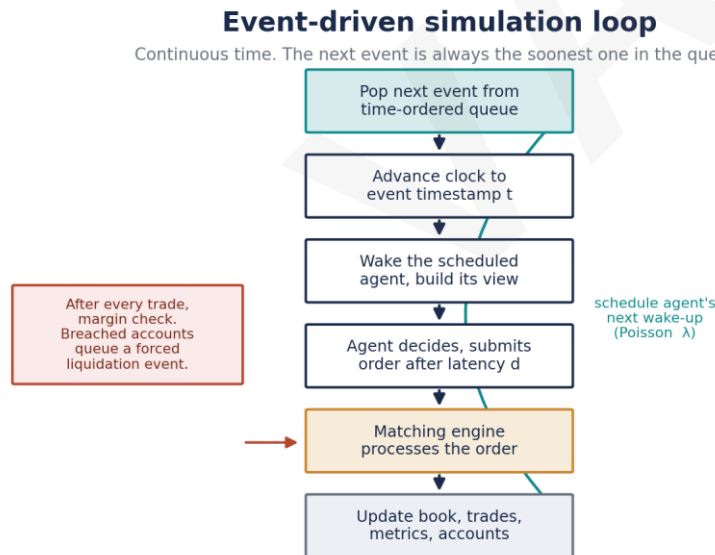


Figure 3. The event loop. The next event is always the soonest one in the queue.

Arrival rates. Each agent has its own rate. Wake ups are drawn from a Poisson process, so the gap between one agent's actions follows an exponential distribution. A market maker might act many times a second. A value investor might act once a day. The mix of rates is part of what shapes the market.

Latency. A delay sits between decision and execution. When an agent decides, its order does not hit the book at once. A delay is added, drawn per agent type. A market maker's delay is tiny. A retail noise trader's is large. Two agents reacting to the same trade can reach the book in a different order because of it, which is realistic and matters for who gets filled.

7 Agent framework

Every agent shares one contract. It observes the market, it decides, and it is told when its orders fill. The base class holds the account, the latency, and the last market view. A strategy is nothing more than an implementation of the decide step, which makes new archetypes cheap to add.

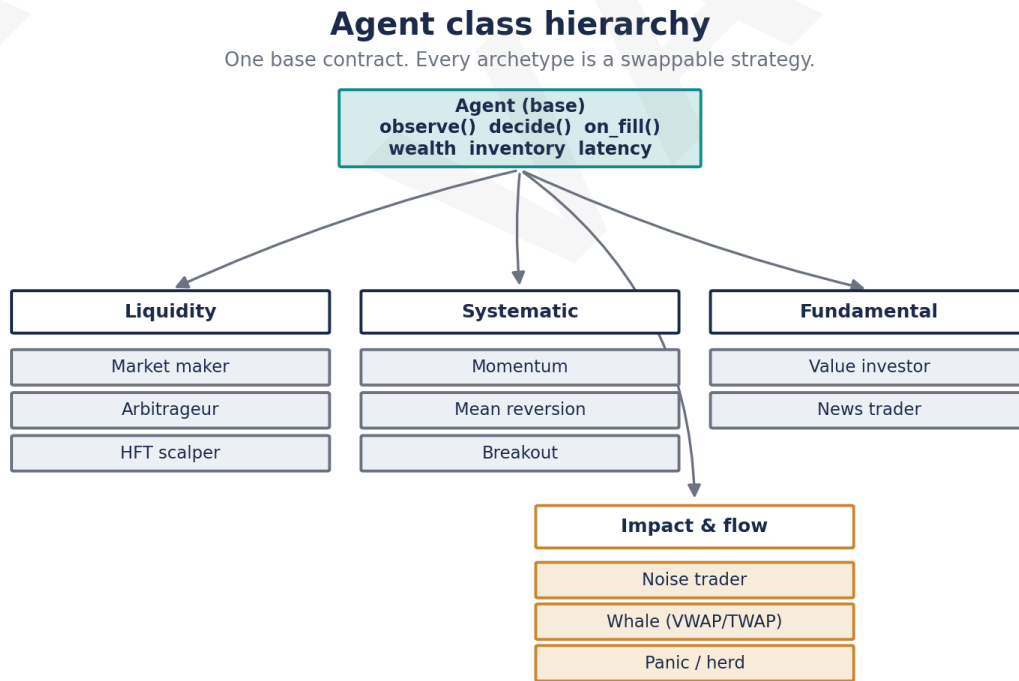


Figure 4. One base contract. Every archetype is a swappable strategy.

Information set. An agent does not see everything. By default it sees the top of the book, the recent trade tape, and its own inventory. Some agents, such as market makers, see the full depth. None see the true value unless their strategy is allowed to, which is what separates a value investor from the rest.

Heterogeneity. Agents of the same type are not clones. Their parameters are drawn from distributions. One momentum trader watches a short window, another a long one. This spread is what produces realistic aggregate behaviour. A market of identical agents behaves nothing like a real one.

Budget. Every agent has finite cash and an inventory limit. It cannot buy what it cannot fund. An agent that loses its capital stops trading, and if it borrowed to get there, it is liquidated. Wealth is not a scoreboard bolted on the side. It feeds back into what the agent can do next.

8 Agent archetypes and their mathematics

Four archetypes make a working market. Noise traders for baseline flow, market makers for liquidity, momentum traders for trends, and value investors to pull the price back towards fundamentals. The other seven are added later. Each strategy below is set out as timing, pricing, and sizing, because those three choices are what define an agent.

8.1 Noise trader

Role. Uncoordinated flow that stands in for retail and for trading on non market signals. It supplies the background variance the other agents feed on

Action. A fair coin picks buy or sell. A fixed probability then splits market orders from limit orders. The default is roughly seventy percent limit and thirty percent market

Pricing. A limit order is placed a small distance from the touch. The offset is drawn from an exponential distribution, so most orders sit close in and a few sit far out

$$P_{bid} = P_{bid}^{best} (1 - \Delta), \quad P_{ask} = P_{ask}^{best} (1 + \Delta), \quad \Delta \sim \text{Exp}(\lambda)$$

Sizing. Order size is drawn from a log normal distribution, which gives the heavy tail seen in real retail flow. Most orders are small, a few are very large

$$V \sim \text{Lognormal}(\mu_{noise}, \sigma_{noise})$$

8.2 Market maker, the Avellaneda and Stoikov model

Role. Quotes both sides of the book at once, earns the spread, and manages the risk of holding stock

A maker that quotes a fixed spread around the mid gets run over when it builds a one sided position. The Avellaneda and Stoikov framework fixes this by shifting its quotes against its inventory. Two quantities do the work.

Reservation price. The price at which the maker is indifferent between holding and selling. It sits below the mid when the maker is long, which makes it keener to sell and slower to buy

$$r(S_t, q) = S_t - q \gamma \sigma^2$$

Here S is the mid price, q is the inventory, gamma is risk aversion, and sigma squared is the variance of the price. A long position pushes the reservation price down.

Optimal spread. The distance between the two quotes. It widens with risk aversion and with volatility, and it narrows when the book is liquid

$$\delta = \gamma \sigma^2 + \frac{2}{\gamma} \ln\left(1 + \frac{\gamma}{K}\right)$$

Here kappa measures how densely orders arrive around the touch. The quotes then sit half a spread either side of the reservation price.

$$P_{bid} = r - \frac{\delta}{2}, \quad P_{ask} = r + \frac{\delta}{2}$$

8.3 Momentum trader

Role. Buys strength and sells weakness. Momentum traders make trends run further and help bubbles form

Signal. The log return over a lookback window of length W. An exponential moving average cross can be used in its place

$$R_t = \ln\left(\frac{S_t}{S_{t-W}}\right)$$

Rule. Above a threshold it buys with a market order. Below the negative of that threshold it sells. Inside the band it does nothing

Sizing. Size scales with the strength of the signal, capped so that a single agent cannot move the whole book on its own

$$V = \min\left(V_{max}, V_{base} \left| \frac{R_t}{\theta} \right| \right)$$

8.4 Value investor

Role. The anchor. It estimates an intrinsic value and leans against the price when the price strays from it

Signal. The gap between the price and its own estimate of value. It buys harder the cheaper the asset looks, and sells as the price runs above value

$$D_t = \frac{P_t - V_t^*}{V_t^*}, \quad \text{trade} \propto -D_t$$

The estimate is noisy and differs across value agents, so they do not all act at the same moment. Their orders are mostly limit, patient, and large. They are the slow money that stops the price drifting away from fundamentals forever.

8.5 The remaining archetypes

These seven are added once the core four work. They are listed here so the design leaves room for them from the start.

Archetype	Role	Timing	Typical order
Arbitrageur	Forces two linked prices back together	On any gap	Market, both books
HFT scalper	Scalps tiny edges, cancels fast	Sub second	Limit, high cancel rate
Mean reversion	Fades stretched moves back to an average	On a band break	Limit
Breakout	Trades a level once it is breached	On breach	Market
News trader	Reacts to value jumps from news	On a news event	Market
Whale	Works a huge order over time by VWAP or TWAP	Scheduled slices	Mixed, impact aware
Panic or herd	Sells into drawdowns, amplifies falls	On a steep fall	Market, aggressive

9 Environment layer

9.1 Fundamental value

The true value of the asset follows a stochastic process. The base is a geometric Brownian motion, which gives a slow drift and steady background noise. On top of that sit occasional jumps, which are news.

$$d \ln V_t = \left(\mu - \frac{1}{2} \sigma_V^2 \right) dt + \sigma_V dW_t + J_t dN_t$$

The drift and the diffusion give the slow path. The jump term, driven by a Poisson counter, is the shock. Value agents and news agents update their estimate when a jump lands.

9.2 News and shocks

At random times the value jumps. A jump is an earnings surprise, a rate decision, or an outside shock. The frequency and the size of jumps are parameters. This is the only exogenous driver in the model. Everything else is endogenous, produced by the agents themselves.

9.3 Circuit breakers

If the price moves more than a set percentage inside a short window, trading halts for a fixed time. Real exchanges do this at seven and thirteen percent. It stops a cascade running the price to zero in a single print, and it gives agents a chance to reset. Modelling it changes how a crash plays out.

9.4 Corporate actions and supply

The float is fixed. Dividends and buybacks can be scheduled, which slowly change value and supply. These are optional and off by default, kept in the design so they can be switched on for a longer horizon study.

10 Frictions

Without friction, agents over trade and the market fills with false high frequency patterns. Four frictions keep behaviour honest.

- **Fees.** a maker taker model. Takers pay a fee, makers earn a rebate. This is what actually pays a market maker, so it has to be modelled or maker behaviour comes out wrong
- **Latency.** the delay before an agent sees a trade, and the delay before its own order lands. Both are set per agent type
- **Slippage.** a market order larger than the top level sweeps down the book, filling at worse and worse prices. This is not a penalty added on top. It falls straight out of matching against real depth

Market impact. Large orders move the price. The temporary part follows the square root law. Impact grows with the square root of the order size relative to daily volume, not in proportion to it

$$\Delta P \approx Y \sigma \sqrt{\frac{Q}{V_{daily}}} \quad (\text{square-root law})$$

11 Leverage, margin, and liquidation

Systemic crashes are almost always amplified by leverage, so the engine models it directly. Some agents trade on borrowed money or sell short. Each carries an account whose equity is cash plus the value of the stock held.

$$E_t = C_t + q_t S_t, \quad \text{liquidate if } \frac{E_t}{|q_t| S_t} < m_{maint}$$

When equity falls below the maintenance fraction of the position, the agent is margin called. The engine then liquidates it with market orders. Those forced sells push the price down, which can margin call the next agent, whose forced sells push it down again. This is how a leverage driven crash is built. It is also the most interesting thing the engine can produce on its own, without any outside shock.

12 Validation framework

This is the part most simulations skip, and it is the most important. A simulation is only realistic if its output looks like a real market. Complexity is not realism. A model with eleven agent types that fails these checks is worth less than one with four that passes them. Before any run is trusted, its price and trade tape are scored against the empirical stylised facts. These are the regularities every liquid market shows, whatever the asset.

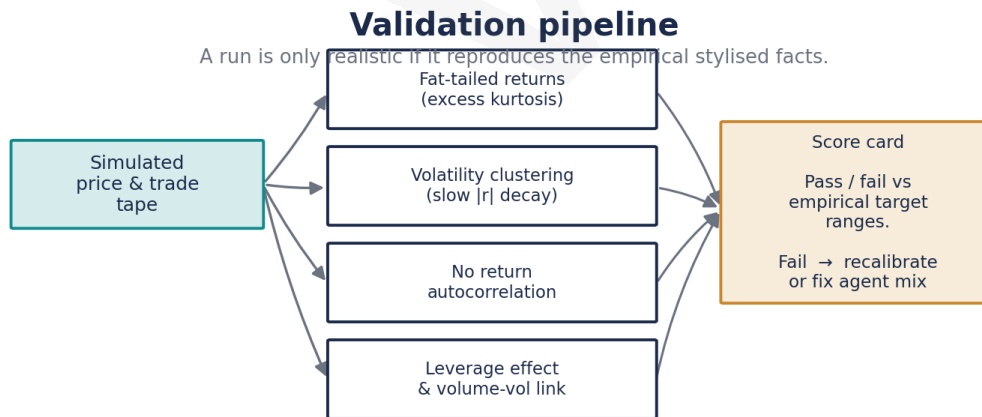


Figure 5. Every run is scored against the stylised facts before it is used.

Fat tails. Returns are not normal. Large moves are far too common for a bell curve. This is measured by the kurtosis of returns, which sits well above three in real data

$$K = \frac{E[(r - \bar{r})^4]}{(E[(r - \bar{r})^2])^2} > 3$$

Volatility clustering. Large moves cluster together. The size of returns stays correlated for a long time, even though the direction does not. This shows up as slowly decaying autocorrelation in absolute returns

$$\text{Corr}(|r_t|, |r_{t+\tau}|) > 0, \text{ decaying slowly in } \tau$$

- **No return autocorrelation.** the direction of the next move is close to unpredictable from the last, so there is no easy linear signal
- **Leverage effect.** volatility rises more on falls than on rises
- **Volume and volatility link.** traded volume moves together with volatility

Each fact has a target range taken from the empirical literature. A run passes or fails on each one. A failing run means the parameters or the agent mix are wrong. It does not mean the world is wrong.

Stylised fact	Rough target	How it is measured
Excess kurtosis of returns	well above 3, often 5 to 30 at short horizons	fourth moment of returns
Volatility clustering	positive, slow decay over many lags	autocorrelation of absolute returns
Return autocorrelation	near zero beyond one lag	autocorrelation of raw returns

Stylised fact	Rough target	How it is measured
Aggregational Gaussianity	returns look more normal at longer horizons	kurtosis by horizon
Leverage effect	negative	correlation of return and future volatility

13 Calibration

The engine has many free parameters. Guessing them is not good enough for a result anyone should trust. Calibration sets them so the output matches chosen targets. The plan runs in three steps of rising ambition.

- **Grid search.** a coarse sweep over the few parameters that matter most, the agent mix and the noise and maker rates. Cheap, and enough to get close
- **Method of simulated moments.** tune parameters until the simulated stylised facts match those of a chosen real asset. This is the main tool
- **Approximate Bayesian computation.** a later option for the cases where a full likelihood is out of reach, giving a spread of plausible parameters rather than one point

14 Analytics and metrics

Every run records a fixed set of outputs. These feed the validation score card, and later any strategy study.

- Price and mid series, with realised volatility
- Spread and depth through time, and order book imbalance
- The full trade tape and traded volume
- **Kyle's lambda.** the price move per unit of signed order flow, a clean measure of how much impact each trade has
- **Profit and loss.** by agent and by archetype, with inventory and a Sharpe ratio
- The cancellation to trade ratio

15 Reproducibility and ensembles

One path proves nothing. It is a single draw from a random process, and it can lie. Every result is an ensemble, many runs with different seeds, reported with a mean and a band around it. Seeds are set once and stored with the run. The same seed gives the same path, so any bug can always be reproduced. A warm up period at the start of each run is discarded, because the book needs time to settle from its empty starting state before the numbers mean anything.

16 Technology and performance

Python is the language, chosen for clarity and for the scientific stack around it. The core loop is the bottleneck. A naive Python order book will not run hundreds of long paths in reasonable time. The plan is a clean pure Python engine first, correct and readable, then a compiled core for the hot matching path once the design is fixed. Numba or Cython does that job. Numpy vectorises the analytics, and runs are stored as parquet for fast reload. Correctness comes before speed, always, but speed is designed for from the start.

17 Project structure

The planned layout on disk. It mirrors the module map, so a reader can move from the architecture to the code without a translation step.

agora/	
engine/	clock, event queue, simulation loop
book/	order book, matching, auctions
orders/	order and trade objects, order types
agents/	base agent and archetypes
market/	environment, value process, news, halts
accounts/	wealth, margin, liquidation
metrics/	analytics, profit and loss
validation/	stylised fact score card
config/	parameters and seeds
run/	ensembles and command line entry
tests/	unit tests per module
notebooks/	exploration and result write ups

18 Roadmap

The build runs in phases. Each phase produces something that works before the next begins. The validation harness arrives early, at phase three, so realism is checked from the first working market rather than bolted on at the end.

Phased build roadmap

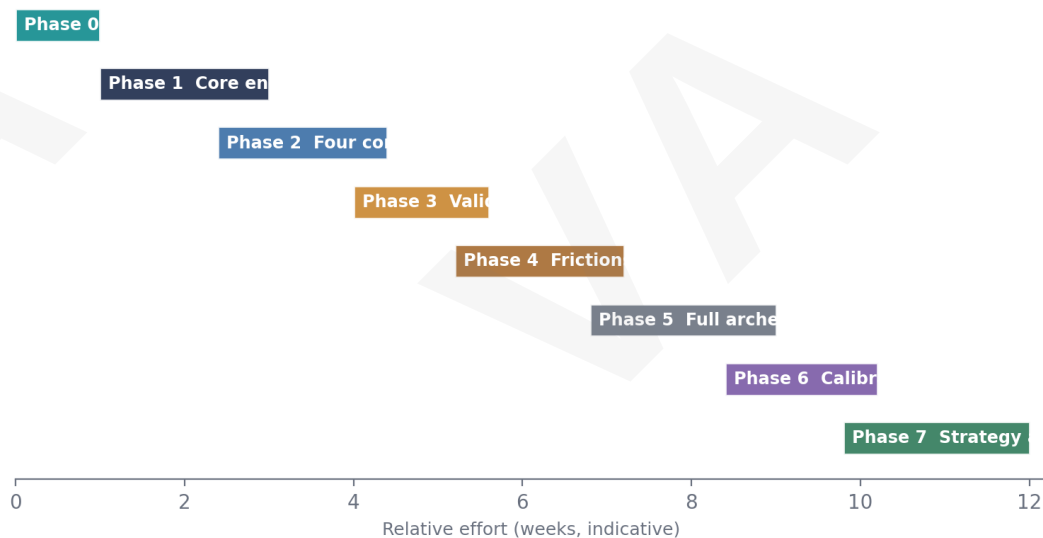


Figure 6. Phased build. Effort is indicative, not a hard schedule.

Phase	Deliverable	Done when
0	Design and specification, this document	the design is agreed
1	Core engine and order book	orders match correctly under test
2	The four core agents	a market runs end to end
3	Validation harness	stylised facts are measured on every run
4	Frictions and margin	fees, latency, and liquidation work
5	Full archetype set	all eleven types are available
6	Calibration	the sim matches a chosen real asset
7	Strategy analysis interface	a strategy can be dropped in and tested

19 Risks and open questions

- **Realism is hard to hit.** reproducing the stylised facts from agents is not guaranteed. It may take several rounds of tuning the mix

- **Performance.** pure Python may prove too slow, forcing the compiled core sooner than planned
- **Calibration is underdetermined.** many parameter sets can fit the same few facts. This limits how much any single fit can be trusted
- **Scope creep.** eleven archetypes and every order type is tempting. The four agent build has to come first
- **Open question on the clock.** the continuous time event queue is chosen over discrete rounds, but the latency model still needs testing to confirm it behaves as intended

20 What to build first

The first working build is deliberately small. One asset. Four agents, being the noise trader, the market maker, the momentum trader, and the value investor. An event driven clock. Limit and market orders. Tick and lot sizes. Wealth limits on every agent. And the validation harness from day one.

The target is simple. Reproduce fat tails and volatility clustering. A build that passes those two facts with four agents is worth more than one with eleven agents that passes none. Everything else in this document is an extension of that core, added only once the core is proven.

Appendix A Parameter reference

The main parameters, their meaning, and a sensible starting value. Defaults are a starting point for calibration, not final numbers.

Symbol	Meaning	Default	Used by
lambda noise	noise trader arrival rate	moderate	noise trader
p market	share of market orders	0.30	noise trader
mu, sigma noise	log normal size parameters	small mean	noise trader
gamma	risk aversion	small positive	market maker
sigma squared	price variance estimate	rolling	market maker
kappa	order arrival density	fitted	market maker
W	momentum lookback window	medium	momentum
theta	momentum trigger threshold	small	momentum
V base, V max	base and capped order size	set	momentum, whale
mu, sigma V	value drift and volatility	low	environment
jump rate, size	news frequency and magnitude	rare, large	environment, news
tick, lot	price and size grid	0.01, 1	engine
maker fee, taker rebate	the fee model	small	engine, maker
m maint	maintenance margin fraction	0.25 range	accounts
breaker percent	halt trigger	7 and 13	environment

Appendix B Notation

Symbol	Meaning
S_t	mid price at time t
q	agent inventory of the asset
r	reservation price of the market maker
δ	the market maker's optimal spread
Δ	the noise trader's price offset from the touch
γ	risk aversion
σ^2	price variance
κ	order book liquidity density
λ	arrival rate of a Poisson process
R_t	log return over the momentum window
θ	momentum trigger threshold
W	momentum lookback window
V^*	intrinsic value estimate
V_t	the true fundamental value process
E_t	account equity
κ (returns)	kurtosis of the return distribution